11/10/25

L-3/T-1/CSE

Date: 24/08/2025

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY, DHAKA

L-3/T-1 B. Sc. Engineering Examinations 2023-2024

Sub: **CSE 309** (Compilers)

Full Marks: 210 Time: 3 Hours

The figures in the margin indicate full marks.

USE SEPARATE SCRIPTS FOR EACH SECTION

SECTION – A

There are **FOUR** questions in this section. Answer Question No. 1 and any **TWO** from the rest. Question No. 1 is **COMPULSORY**.

1. (a) Consider the following three files written in C programming language: **(10)**

main.c

```
1 extern int foo(int x);  // Defined in libfoo
2 extern int bar(int x);  // Defined in libbar
3
4 int main() {
5     return foo(5) + bar(3);
6 }
```

libfoo.c

```
1 extern int baz(int x);  // Defined in libbar
2
3 int foo(int x) {
4     return baz(x + 1);
5 }
```

libbar.c

```
1 int baz(int x) {
2     return x * 2;
3 }
4
5 int bar(int x) {
6     return x + baz(x);
7 }
```

Assume these files are compiled into:

- `main.o` (relocatable object file for main),
- `libfoo.o` (shared library),
- `libbar.o` (shared library).

Explain the terms relocatable machine code, linker, and loader from the perspective of compilation and execution of the above program.

Contd P/2

(86)

CSE 309
(Contd ... Q. No. 1)

(b) You are designing the lexical analyzer for a complier. The language includes the arithmetic operators +, –, * and /. Consider the following options for defining tokens for these operators: **(12)**

1. Distinct tokens: Define completely different tokens for each operator:
   - `+` → `TOK_PLUS`
   - `-` → `TOK_MINUS`
   - `*` → `TOK_MUL`
   - `/` → `TOK_DIV`
2. Grouped tokens: Define one token for additive operators and another for multiplicative operators:
   - `+, -` → `TOK_ADD_OP`
   - `*, /` → `TOK_MUL_OP`
3. Single token: Define a single token for all arithmetic operators:
   - `+, -, *, /` → `TOK_ARITH_OP`

Which approach would you recommend and why? Justify your answer with specific trade-offs between implementation complexity and language design requirements.

(c) Consider the grammar given below: **(13)**

$S \rightarrow aB \mid bA$
$A \rightarrow a$
$B \rightarrow b$

By carrying out rightmost derivations, find out whether the LR(1) items shown in the following table are valid for the corresponding viable prefixes shown in the same table.

Viable Prefix	Item
a	$[S \rightarrow a \cdot B, \$]$
b	$[S \rightarrow b \cdot A, \$]$
b	$[A \rightarrow \cdot a, \$]$
ab	$[B \rightarrow \cdot b, b]$
baa	$[A \rightarrow A \cdot a, \$]$
ab	$[B \rightarrow b \cdot, \$]$
ba	$[A \rightarrow a \cdot, \$]$
a	$[A \rightarrow \cdot a, a]$
b	$[S \rightarrow b \cdot A, a]$

Contd P/3

CSE 309

2. (a) Elaborate on the advantages of separating lexical analysis form syntactic analysis, with a particular focus on how this separation contributes to the simplification and clarity of complier design. **(6)**

(b) Is the use of intermediate representations strictly necessary in compiler design? Discuss in detail the potential benefits of incorporating intermediate representations within a compiler, supporting your explanation with relevant examples. **(9)**

(c) How does the buffer size of buffer pairs used in lexical analysis influence the characteristics and attributes of a programming language? Provide a detailed discussion. **(5)**

(d) Eliminate left-recursion for the following grammar: **(15)**

$$S \rightarrow Sa \mid Bb \mid c$$
$$B \rightarrow Bd \mid Se \mid Cf \mid g$$
$$C \rightarrow Ca \mid Bh \mid Sc \mid d$$

3. (a) Construct procedure based recursive-descent parsers, starting with each of the following grammars: **(10)**

i. $S \rightarrow S(S)S \mid \in$
ii. $S \rightarrow 0S1 \mid 01$

(b) Write down an *LL(0)* grammar containing at least three production rules. Justify why your answer is correct. **(6)**

(c) Consider grammar given below: **(14+5)**

$$S \rightarrow A - B)$$
$$A \rightarrow B \mid a)$$
$$B \rightarrow aB \mid a$$

i. Construct the LR(1) automata for the grammar.

ii. Inspect the items carefully and comment on whether the grammar is SLR(1) or not.

4. (a) Compare the automation-based bottom-up parsers in terms of automation used, number of states and parsing capability in a nicely formatted table. **(6)**

(b) When an LALR parser is constructed by merging states form a CLR parser, does it revert to being the same as an SLR parser? Explain clearly. **(9)**

(c) Argue carefully, with relevant examples, whether it is possible that when we replace all states having the same core of an LR(1) grammar with their union to build an LALR parser, is it possible that the resulting union will have a conflict. **(10)**

Contd P/4

CSE 309
(Contd ... Q. No. 4)

(d) For the grammar: **(10)**

1. $S \rightarrow SS+$
2. $S \rightarrow SS*$
3. $S \rightarrow a$

Here is the SLR (1) parse table:

Table for Question 4(d)

State	ACTION				GOTO		
	a	+	*	$	S	A	B
0	s2				1		
1				acc			
2	s4	r3	r3	r3			3
3				r1			
4	s4	r3	r3	r3			5
5		s7	s8			6	
6	s4	r3	r3	r3			9
7	r4			r4			
8	r5			r5			
9		r2	r2	r2			

Show the parsing steps in a table on the input aa*a+.

SECTION – B

There are **FOUR** questions in this section. **Answer Q. No. 5** and any **TWO** from the rest.
Question No. 5 is **COMPULSORY**.

5. (a) Consider the flow graph below derived from some three-address code. Optimize the code by discovering all possible semantic preserving transformations. For each of the transformations used, specify the name of the transformation. **(16)**

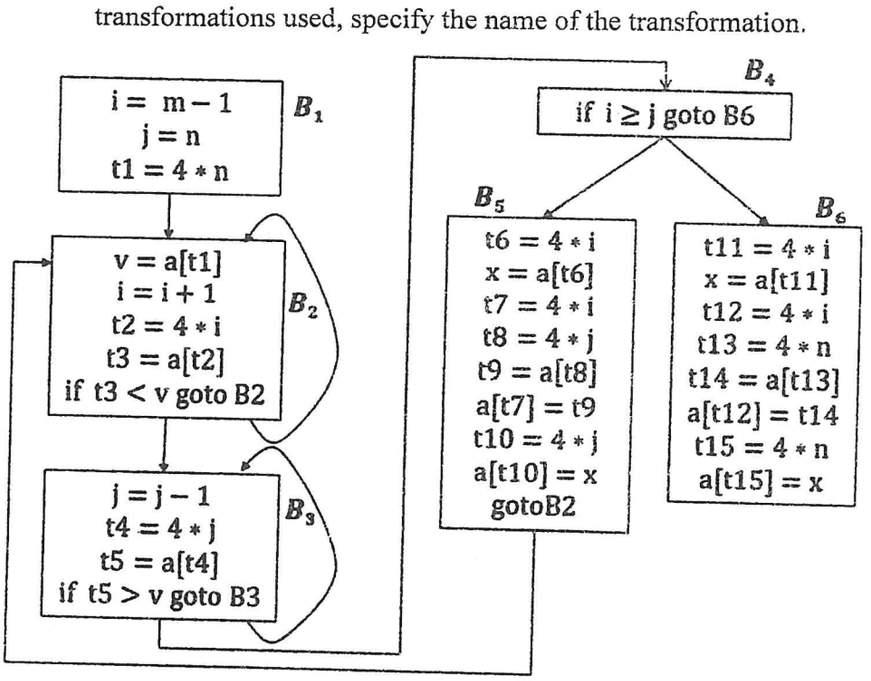


Figure for Question 5(a)

Contd P/5

CSE 309
(Contd ... Q. No. 5)

(b) Consider below a 'C' code fragment and the corresponding assembly code generated by some 'gcc' compiler using –O2 optimization switch. Now, identify and explain the optimizations that have been performed by the compiler for the given 'C' code. Will the same optimization be performed if the array 'arr[10]' is declared as a global variable? Justify your answer. **(9)**

Program written in C

```
#include<stdio.h>
int main() {
  int arr[10];
  int n;
  scanf("%d",&n);
  for (int i=0; i<n-1;i++){
    for (int j=0; j<n-i-1; j++){
      if (arr[j] > arr[j+1]){
         int temp = arr[j];
         arr[j]=arr[j+1];
         arr[j+1]=temp;
       }
    }
  }
  return 0;
}
```

Assembly code generated by gcc

```
.LC0:
        .string "%d"
"main":
        sub     rsp, 24
        mov     edi, OFFSET
FLAT:.LC0
        xor     eax, eax
        lea     rsi, [rsp+12]
        call    __isoc99_scanf
        xor     eax, eax
        add     rsp, 24
        ret
```

(c) How can DAGs (Directed Acyclic Graph) be used to optimize basic blocks? Illustrate using the example basic block below. Assume d and f are not live on exit from the block. **(10)**

```
d = b * c
e = a[i]
f = e + d
b = a[i]
a[j] = b
c = a[i]
```

6. (a) What is the difference between SDD (Syntax-Directed Definition) and SDT (Syntax-Directed Translation Scheme)? Which of the two is suitable for implementation? Explain. Name two classes of SDD that allow implementation during parsing. **(10)**

(b) The following SDT prints the prefix form of an expression, i. e., performs infix to prefix translation. Explain why it is not possible to implement the SDT during either top-down or bottom-up parsing. Describe, then, how such an SDT can be implemented. **(4+9)**

$L \rightarrow En$

$E \rightarrow \{print('+');\}\ E_1 + T$

$E \rightarrow T$

$T \rightarrow \{print('*');\}\ T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow digit\ \{print(digit.level);\}$

Contd P/6

CSE 309
(Contd ... Q. No. 6)

(c) What kinds of data are usually stored in an activation record? What are 'calling sequence' and 'return sequence' in the context of a procedure call? Describe some of the guiding principles while designing calling sequences and the layout of activation records. **(12)**

7. (a) Briefly describe two applications of types in a program. Consider the following grammar related to variable declarations. **(4+8)**

$P \rightarrow D$
$D \rightarrow T\,\mathbf{id};D \mid \epsilon$

Now, convert the above to an SDT (Syntax-Directed Translation Scheme) to save each declared variable in the symbol table along with its type and offset information. The symbol T in the above grammar represents type of **id** and it has two synthesized attributes *type* and *width*. Based on your SDT, what will be the offset of each of the variables in the below declarations.

int a; **int** b; **float** f;

Assume that the size of **int** is 2 byte and size of **float** is 4 byte.

(b) Consider the following SDD for generating three-address codes using the fall-through technique. **(12)**

Production	**Semantic Rules**
$S \rightarrow$ **if** (B) S_1	*B.true* = *fall* *B.false* = S_1.*next* = *S.next* *S.code* = *B.code* || S_1.*code*
$B \rightarrow B_1 \,\|\|\, B_2$	B_1.*true* = **if** *B.true* ≠ *fall* **then** *B.true* **else** *newLabel*() B_1.*false* = *fall* B_2.*true* = *B.true* B_2.*false* = *B.false* *B.code* = **if** *B.true* ≠ *fall* **then** B_1.*code* || B_2.*code* **else** B_1.*code* || B_2.*code* || *label*(B_1.*true*)

Now add rules to the SDD above for the following two productions using the same fall-through technique

(*i*) $B \rightarrow B_1 \,\&\&\, B_2$
(*ii*) $B \rightarrow E_1$ **rel** E_2

Assume all the symbols, attributes, functions, tables, and notations have their usual meanings.

(c) Based on the SDD of Question 7(b), draw the annotated parse tree for the statement below and find out the code generated by the SDD. Operators have their usual precedence and associativity. **(11)**

```
if( x > 100 || x > 200 && x != y ) x = 0 ;
```

Contd P/7

CSE 309

8. (a) What are 'temporal locality' and 'spatial locality' in the context of a program execution? How does a garbage collector (GC) attempt of improve these two types of 'locality of reference'? Each garbage collection algorithm starts with a 'root set'. Which objects constitute the 'root set'? Consider below the pseudocode for Cheney's 'copying collector' from your text. Complete the code by defining the function '*LookupNewLocation*'. **(15)**

```
1)  CopyingCollector () {
2)        for (all objects o in From space) NewLocation(o) =NULL;
3)        unscanned = free = starting address of To space;
4)        for (each reference r in the root set)
5)              replace r with LookupNewLocation(r);
6)        while (unscanned ≠ free) {
7)              o = object at location unscanned;
8)              for (each reference o.r within o)
9)                    o.r = LookupNewLocation(o.r);
10)             unscanned = unscanned + sizeof(o);
          }
    }
```

(b) Consider the following register interference graph (RIG) derived from the flow graph of some three address code. What do the vertices and edges denote in such a RIG? Using the RIG, explain how the problem of global register allocation can be formulated as a graph coloring problem. Subsequently, find register allocation for the given program using Chaitin's algorithm. Assume three physical registers are available. Describe how the algorithm work and illustrate all steps of the algorithm for the given RIG. Make any other justified assumptions as required. What is a 'spill' in the context of the algorithm? Explain briefly how spilled variables are handled. **(20)**

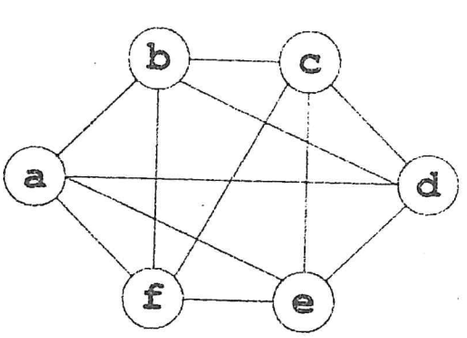


Figure for Question 8(b)

13/5/2024

L-3/T-1/CSE Date: ~~31/03/2024~~

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY, DHAKA

L-3/T-1 B. Sc. Engineering Examinations 2022-2023

Sub: **CSE 309** (Compiler)

Full Marks: 210 Time: 3 Hours

The figures in the margin indicate full marks.

USE SEPARATE SCRIPTS FOR EACH SECTION

SECTION – A

There are **FOUR** questions in this section. Answer to **Question no. 1 (one) is Compulsory**. Answer any **TWO** questions from Questions 2-4.

1. [COMPULSORY]

(a) Appraise the differences between bytecode and machine language. **(3)**

(b) The partial contents of a buffer in a lexical analyzer is **(6)**

```
cse123--++12.45==␣1␣
```

Where ␣ indicates a space. At the current moment, both `lexemeBegin` and `forward` pointers are on c of cse (indicated as position 0). From the perspective of C programming language, write down the positions of the pointers in the table below when this string is scanned. Also, indicate any token identified in the third row. Not all the cells in this row will get populated. There may be more columns than needed in this table.

`lexemeBegin`													
`forward`													
`Token`													

⋮

`lexemeBegin`													
`forward`													
`Token`													

(c) Given below are several statements/entity names related to the linker and loader. Determine which of the statements are related to the linker and which of the statements are related to loader. Only the serial numbers will suffice. You can assign a serial number to either the linker or loader, but not to both. **(5)**

(i) Object files.

(ii) Accountable for loading programs and libraries.

(iii) Load executable files to memory.

(iv) Produce executable files.

(v) Allocate address to executable files.

(vi) Accountable for managing objects in the program's space.

(vii) Setting up references that are utilized in the program.

Partial solutions provided below as examples.

Linker: i.

Loader: ii.

Contd P/2

CSE 309

Contd ... Q. No. 1

(d) When you carry out left factoring of a grammar, argue carefully that the grammar produced generates the same language as the original grammar. **(6)**

(e) Write a procedure based predictive parser for the following grammar keeping the grammar unaltered: **(5)**

$$E \rightarrow E + F \mid E\text{-}F \mid F$$

$$F \rightarrow \mathbf{NUM} \mid \mathbf{ID}(E)$$

Point out any possible flaw of the parser you have constructed.

(f) What will be the general appearances of the LL(0) grammars? Provide relevant examples. Comment on the applicability of these grammars for useful parsing. **(5)**

(g) For a certain context free grammar, there is exactly one production rule for each nonterminal. What kind of parser do you suggest for this grammar keeping optimality in mind? Explain clearly. **(5)**

2. (a) Why are the universal parsing methods not used in production compilers? **(3)**

(b) Elucidate carefully how the SLR(0) parsing table will be different from an SLR(1) parsing table. **(5)**

(c) Eliminate left recursion from the grammar: $A \rightarrow Ba \mid Aa \mid c,\ B \rightarrow Bb \mid Ab \mid d.$ **(10)**

(d) Explain how the separation of the analysis portion of a compiler into lexical analysis and parsing (syntax analysis) phases, contributes to the adoption of a new language to an already existing compiler. **(5)**

(e) For the grammar given below, find the FOLLOW sets of the non-terminals. **(12)**

$S \rightarrow aABe$

$A \rightarrow B \mid \epsilon$

$B \rightarrow bB \mid Cd$

$C \rightarrow cC \mid \epsilon$

$D \rightarrow dD \mid \epsilon$

3. (a) In a certain cell of a predictive parsing table, there are two entries of the form $A \rightarrow \alpha$ and $A \rightarrow \epsilon$. Explain what this means from the context of the FIRST and FOLLOW sets. **(7)**

(b) Consider the context free grammar given below: **(10)**

$S \rightarrow Aab \mid aS \mid bc$

$A \rightarrow aacd \mid bef$

$B \rightarrow bef \mid \epsilon$

Contd P/3

CSE 309

Contd ... Q. No. 3(b)

If we consider the above grammar to be LL(k), what will be the value of k? Answer with necessary analysis of each of the productions.

(c) Explain why LL(1) parsing is associated with left-most derivations of parse trees. **(6)**

(d) Show that the grammar: $S \rightarrow AaAb \mid BbBa$, $A \rightarrow \epsilon$, $B \rightarrow \epsilon$, is LL(1) but not SLR(1). **(12)**

4. (a) Explain how empty production rules (epsilon productions) are handled in procedure based predictive parsers. **(5)**

(b) For the grammar $S \rightarrow SS+ \mid SS* \mid \alpha$, identify the handles for parsing the right-sentential form aaa*a++. Show your computations. **(10)**

(c) We say that a configuration $[A \rightarrow \beta_1 . \beta_2, \alpha]$ is valid for a viable prefix $\alpha\beta_1$ if there is a derivation $S' \underset{rm}{\overset{*}{\Longrightarrow}} \alpha Aw \underset{rm}{\Longrightarrow} \alpha\beta_1\beta_2 w$. Why? **(10)**

(b) We have an LR(1) grammar. If we formulate an LALR(1) grammar from this, do you think that the resulting union will have a parsing-action conflict? Justify your answer. **(10)**

SECTION – B

There are **FOUR** questions in this section. Answer to **Question no. 5 (five) is Compulsory.** Answer any **TWO** questions from Questions 6-8.

(All the symbols have their usual meanings in the context of compilers unless explicitly mentioned)

5. (a) Consider the three-address code below and answer the following questions. **(10+15)**

(i) Identify the basic blocks and construct the flow graph for the given code. Give each block an identifier and replace the label references in the statements with the corresponding identifiers.

(ii) Optimize the code by discovering all possible semantic preserving transformations. For each of the transformations used, specify the name of the transformation.

```
 1    L1: i = 0
 2    L3: t1 = n - 1
 3        iffalse i < t1 goto L2
 4    L4: j = i + 1
 5    L5: iffalse j < n goto L6
 6    L7: t2 = i * 8
 7        t3 = a [ t2 ]
 8        t4 = j * 8
 9        t5 = a [ t4 ]
10        iffalse t3 > t5 goto L8
11    L9: t6 = i * 8
12        p = a [ t6 ]
13    L10: t7 = i * 8
14         t8 = j * 8
15         t9 = a [ t8 ]
16         a [ t7 ] = t9
17    L11: t10 = j * 8
18         a [ t10 ] = p
19    L8: j = j + 1
20        goto L5
21    L6: i = i + 1
22        goto L3
23  L2:
```

CSE 309

Contd ... Q. No. 5

(b) Consider a 'C' code fragment and the corresponding assembly code generated by some 'gcc' compiler using –O2 optimization switch. Now, identify and expain the optimizations that have been performed by the compiler for the given 'C' code. **(10)**

Program written in C

```
#include<stdio.h>
int a[50], b[50],x,y,z,k,n;
int factorial(int n)
{
    int fact;
    for(int i=n;i>=0; i--)
        fact=fact*i;
    return fact;
}
int main(){
    int t;
    t=y;
    y=z;
    z=t;
    x=y^y;
    if(x) n=10;
    else n=20;
    k=factorial(n);
    for (int j=0;j<50;j++)
        for(int i=0;i<100;i++)
                a[j]*=b[j];
    return 0;
    for (int i=0;i<50;i++)
        a[i]=b[i];
}
```

Assembly code generated by gcc

```
factorial(int):
    xor     eax, eax
    ret
main:
    mov     eax, DWORD PTR y[rip]
    mov     edx, DWORD PTR z[rip]
    xor     esi, esi
    mov     DWORD PTR x[rip], 0
    mov     DWORD PTR n[rip], 20
    mov     DWORD PTR y[rip], edx
    mov     DWORD PTR z[rip], eax
    mov     DWORD PTR k[rip], 0
.L4:
    mov     eax, DWORD PTR a[rsi]
    mov     ecx, DWORD PTR b[rsi]
    mov     edx, 100
.L5:
    imul    eax, ecx
    imul    eax, ecx
    sub     edx, 2
    jne     .L5
    mov     DWORD PTR a[rsi], eax
    add     rsi, 4
    cmp     rsi, 200
    jne     .L4
    xor     eax, eax
    ret
```

6. (a) Below is a grammar for simple expressions involving only numbers and the operators '+' and '*' such as 2*(3+4)*5. Write a postfix Syntax-Directed Translation Scheme (SDT) to compute the DAG (Directed Acyclic Graph) for an expression represented by the given grammar. Explain how your SDT would detect common subexpressions. **(12)**

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

$F \rightarrow num$

CSE 309

Contd ... Q. No. 6

(b) Consider the following E-productions from an SDT for translating infix expressions to postfix notation. Transform the SDT such that it can be implemented during top-down parsing. Show the correctness of your SDT by computing the output for the expression 2+3+4. **(5+4)**

$E \rightarrow E_1 + T$ {print('+'); }

$E \rightarrow T$

(c) What is the use of a φ-function is SSA (Static Single-Assignment) form. Explain with a suitable example. **(8)**

(d) Write the three-address code for the statement **(6)**

```
x = a[b[c[j]]]
```

Assume that- (i) the arrays a, b and c are declared as, int a[5], b[3], c[3]; types and widths of the arrays have correctly been saved in the symbol table, (ii) i and x are integers, and (iii) the size of an integer is 2 byte.

7. (a) Give an outline of syntax-directed translations of *switch*-statements of the following form. Accordingly, show the three-address translation of a *switch*-statement. Briefly discuss how the n-way branch in a *switch*-statement can be evaluated. **(10+5)**

```
switch ( E ) {
        case V1: S1
        case V2: S2
                 ...
        case Vn-1: Sn-1
        default: Sn
}
```

(b) Write down the three-address code for the statement below avoiding redundant gotos. Assume &&, || are left associative and precedence order is && then ||. **(5)**

```
if ( x > 20 && (x < 10 || x ! = y ))
   x = 0;
else x = 1;
```

(c) Illustrate, using an example scenario, the difference between basic *mark-and-compact* and *Cheney's copying collector* algorithms for garbage collection. Specify the running-times of both the algorithms. **(12+3)**

Contd P/6

CSE 309

8. (a) Consider that the following code computes the binomial coefficient recursively. **(10)**

```
int binCoeff(int n, int r)
{
    if (r == 0 || r == n) //base cases
        return 1;

    return binCoeff(n - 1, r - 1) + binCoeff(n - 1, r);
}

int main()
{
    cout << binCoeff(5, 3);
    return 0;
}
```

Now, answer the questions below.

(i) Show the complete activation tree for it.

(ii) What is the maximum number of activation records that will reside in the stack during execution of the program? Explain your answer using your activation tree.

(iii) What does the control stack and its activation records look like when the base cases is satisfied for the first time and the function is about to return. Show only the arguments in an activation record.

(b) Determine the liveliness and next-use information for each statement in the basic block below. Assume t, u and v are temporary variables and the remaining variables are live on exit from the block. **(8)**

```
t = a - c
u = d - b
t = a + b
a = u + t
d = t + v
```

(c) Illustrate the 'linear scan' register allocation algorithm using the following three-address code as an example. Assume three physical registers are available. Make any other justified assumptions as required. What is the advantage of this algorithm? **(15+2)**

```
1       a=b+c
2       d=d-b
3       e=a+f
4       ifFalse e goto L1
5       f=a-d
6       goto L2
7   L1:
8       b=d+f
9       e=a-c
10  L2:
11      b=d+c
```

L-3/T-1/CSE **Date: 30/03/2023**

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY, DHAKA

L-3/T-1 B. Sc. Engineering Examinations 2021-2022

Sub: **CSE 309** (Compiler)

Full Marks: 210 Time: 3 Hours

The figures in the margin indicate full marks

USE SEPARATE SCRIPTS FOR EACH SECTION

SECTION – A

There are **FOUR** questions in this section. Answer **Q. No. 1** and any **TWO** from the rest.

Question No. 1 is COMPULSORY.

1. (a) Consider the C code shown below contained in two separate files. Discuss in detail the issues faced by the linker and the loader when he program is compiled and then executed. How will these issues be resolved? Explain. **(13)**

main.c

```
1 #include <stdio.h>
2 void main()
3 {
4   .......;
5   int i;
6   i = subfunction();
7   exit();
8 }
```

subcode.c

```
1 void subfunction()
2 {
3   int i, n = 0;
4   for (i = 1; i <= 50; i++)
5     {
6       n = n + i;
7     }
8   return n;
9 }
```

(b) The partial contents of a buffer in lexical analyzer is `dhaka␣+++␣123.456␣`, where ␣ indicates a space. At the current moment, both the `lexemeBegin` and `forward` pointers are on `d` at left. From the perspective of the C programming language, describe clearly the pointer movements and recognition of tokens that will happen. **(10)**

Contd P/2

CSE 309

Contd.... for Q. No. 1

(c) Justify with proper reasons the acceptability (or unacceptability) of the each of the following statements at any valid state in shift-reduce parsing. **(12)**

(i) Viable prefixes appear only at the bottom of the stack and not inside.

(ii) Viable prefixes appear only at the top of the stack and not inside.

(iii) The stack contains only a set of viable prefixes.

(iv) Viable prefixes do not continue past the right end of the rightmost handle of that sentential form.

2. (a) Among the reasons why the analysis portion of the compiler is normally separated into lexical analysis and parsing (syntax analysis) phases, simplicity of design is the most important consideration — explain and justify. **(10)**

(b) In the LR-parsing algorithm, if ACTION$[s_m, a_i]$ = reduce $A \rightarrow \beta$, describe in detail how exactly this action is carried out. The symbols have got their meaning used in the class. What is the relationship of β with the stack content? **(10)**

(c) Enumerate and explain the classes that cover most or all of the tokens in the programming languages. **(7)**

(d) What is the one-liner rule to determine whether a token will require an attribute or not? Are all the token attributes single values? Explain. **(5)**

(e) Is input buffering essential in lexical analysis? Explain. **(3)**

3 (a) Explain clearly with examples, for a production rule like, $X \rightarrow Y_1 Y_2 ... Y_k$, when ϵ is included in FIRST(X). **(8)**

(b) Describe what exactly happens during parsing when a cell of the predictive parsing table has an empty production rule like, $A \rightarrow \epsilon$. **(6)**

(c) What are the key decisions during top-down and bottom-up parsing? **(6)**

(d) Consider the following grammar: **(10)**

$A \rightarrow kXYz$

$X \rightarrow Xwr \mid w$

$Y \rightarrow g$

For the string `kwwrgz`, draw the parse tree and hence identify the handles for bottom-up parsing.

(e) In shift-reduce parsing, the handle will always eventually appear on top of the stack, never inside — justify this statement for the problem in (d). **(5)**

CSE 309

4. (a) Eliminate left recursion from the grammar: **(12)**

$$S \rightarrow SBr \mid a$$
$$B \rightarrow Cs \mid b$$
$$C \rightarrow St \mid c$$

(b) For the grammar shown below, find the LR(1) canonical items. **(10)**

$$S \rightarrow aAd \mid bBd \mid aBe \mid bAe$$
$$A \rightarrow c$$
$$B \rightarrow c$$

(c) From the LR(1) canonical items in Question 4(b) construct the CLR parsing table. **(8)**

(d) Comment on the similarity and dissimilarity between the sequences of shifts and reductions for the LR and LALR parsers. **(5)**

SECTION – B

There are **FOUR** questions in this section. Answer any **THREE** questions.
All the symbols have their usual meanings in the context of compilers unless explicitly mentioned.

5. (a) What is a dependency graph in the context of Syntax-Directed Definition (SSD)? How is a dependency graph constructed? What is the application of a dependency graph? **(4+4+3=11)**

(b) The following Syntax-Directed Translation Scheme (SDT) computes the value of a string of 0's and 1's interpreted as a positive, binary integer. **(10)**

$$B \rightarrow B_1 0 \{B.val = 2 \times B_1.val\}$$
$$\mid B_1 1 \{B.val = 2 \times B_1.val + 1\}$$
$$\mid 1 \quad \{B.val = 1\}$$

Rewrite this SDT so that the underlying grammar is not left recursive, and yet the same value of *B.val* is computed for the entire input string.

(c) Give an example of an L-attributed SDD on an LR grammar. Is it possible to handle every LR grammar and L-attributed SDD bottom up? Give an intuitive argument in support of your answer. **(6+8=14)**

6. (a) Illustrate two distinctive aspects that distinguish Static Single-Assignment (SSA) from three-address code. How does SSA facilitate code optimization? Explain with an example. **(8+5=13)**

(b) Consider the following SDD for generating three-address codes using the fall-through technique. **(14)**

Contd P/4

CSE 309

Contd.... for Q. No. 6(b)

Production	Semantic Rules
$S \rightarrow$ **if** (B) S_1	$B.true = fall$ $B.false = S_1.next = S.next$ $S.code = B.code \mid\mid S_1.code$
$B \rightarrow B_1 \mid\mid B_2$	$B_1.true =$ **if** $B.true \neq fall$ **then** $B.true$ **else** $newLabel()$ $B_1.false = fall$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code =$ **if** $B.true \neq fall$ **then** $B_1.code \mid\mid B_2.code$ **else** $B_1.code \mid\mid B_2.code \mid\mid label(B_1.true)$

Now add rules to the SDD above for the following two productions using the same fall-through technique.

(i) $B \rightarrow B_1$ && B_2

(ii) $B \rightarrow E_1$ rel E_2

Assume all the symbols, attributes, functions, labels, and notations have their usual meanings.

(c) Write down a translation scheme for generating code using *backpatching* for the flow-of-control statement yielded by the following production. You may add marker symbols in the production as required; specify the semantic actions corresponding to the marker symbols. Make any other justified assumptions, as necessary. **(8)**

$S \rightarrow \text{for}(S_1; B; S_2)S_3$

7. (a) Which condition must procedure calls or *activations* of procedure satisfy to enable stack allocation? Explain. Give an example of a programming language feature that violates the condition and thus cannot be implemented with purely a runtime stack. What are calling and return sequences in the context of procedure calls? **(6+6+5=17)**

(b) Consider the network of objects in the figure below. Assume, X denotes the *root set*. Suppose at some point in time, the pointer $A \rightarrow C$ is deleted. Then we execute Cheney's copying garbage collection algorithm on the network. Also, suppose that, **(18)**

(i) Each object has size 100 bytes,

(ii) The unscanned list is managed as a queue, and when an object has more than one pointer, the reached objects are added to the queue in alphabetical order,

(iii) The *From* semispace starts at location 0, and the *To* semispace starts at location 10,000, and

(iv) Initially, all the objects in the heap are arranged in alphabetical order starting at byte 0.

What is the value of *NewLoction(o)* for each object o that remains after garbage collection? Draw the heaps before and after running garbage collection, showing- (i) locations of the allocated objects, (ii) references running among the objects. What is the time complexity of the algorithm?

Contd P/5

CSE 309

Contd.... for Q. No. 7(b)

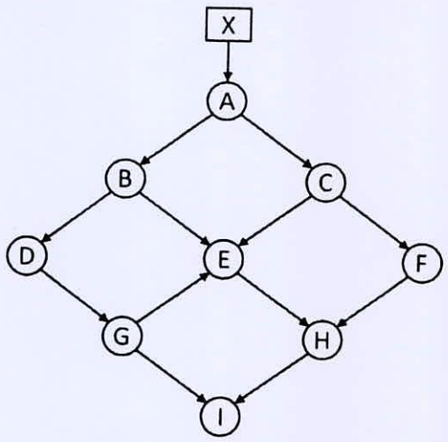


Figure for Question 7(b)

8. (a) Optimize the following three-address code by eliminating common subexpressions, performing reduction in strength on induction variables, and eliminating all the induction variables as possible. **(10)**

```
    dp = 0
    i = 0
L:  t1 = i*8
    t2 = A[t1]
    t3 = i*8
    t4 = B[t3]
    t5 = t2*t4
    dp = dp + t5
    i = i + 1
    if i<n goto L
```

(b) Given the flow graph of a program, formulate the problem of global register allocation as a graph coloring problem. Show detailed formulation steps using the flow graph in the figure below as an example. Subsequently, find a register allocation for the given program using *Chaitin's* algorithm. Assume that three physical registers are available. Illustrate all steps of the algorithm. **(13+12=25)**

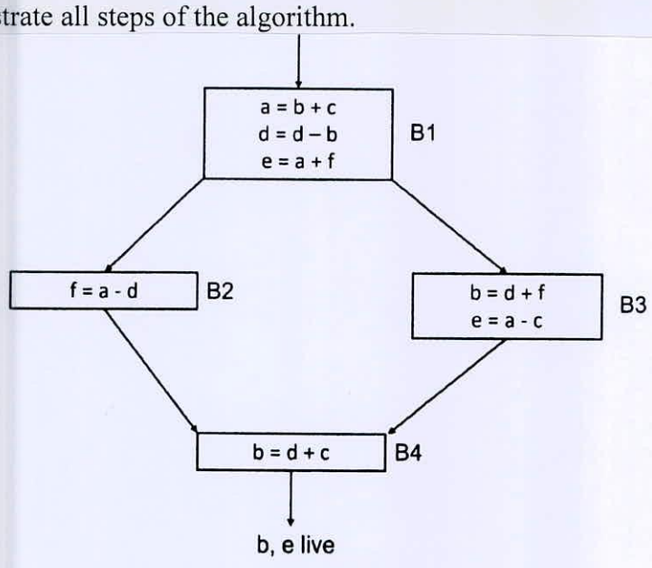


Figure for Question 8(b)

L-3/T-1/CSE Date: 25/10/2022

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY, DHAKA

L-3/T-1 B. Sc. Engineering Examinations 2020-2021

Sub: **CSE 309** (Compiler)

Full Marks: 210 Time: 3 Hours

USE SEPARATE SCRIPTS FOR EACH SECTION

The figures in the margin indicate full marks

SECTION – A

There are **FOUR** questions in this section. Answer any **THREE**.

All the symbols have their usual meanings in the context of compilers unless explicitly mentioned.

1. The following grammar generates a list numbers of the form- **(12)**

$\{num_1, num_2, num_3, \ldots, num_n\}$

where the terminal **num** represents any constant integer number.

$L \rightarrow$ '{' N '}'

$N \rightarrow N$ ',' **num**|**num**

(a) Design a syntax-directed definition (SDD) based on this grammar, which will compute and print for a given list of numbers –

(i) the index of the maximum number and

(ii) the count of numbers whose values are greater than their indices.

Assume the leftmost number has the index **0**.

For examples, for the input {3, 6, 1, 2, 5}, (i) the index of the maximum number is **1**, *and (ii)* the count of numbers whose values are greater than their indices is **3**.

(b) For the SDD you have designed in the answer to Question 1(a), draw the annotated parse tree for the list- {3, 6, 1, 2, 5} **(6)**

(c) Convert the SDD you have designed in the answer to Question 1(a) to an SDT. Can the SDT be implemented during top-down parsing? If not, transform the SDT such that it is implementable during top-down parsing. **(8)**

(d) Define Postfix SDT. Briefly des how Postfix SDT's can be implemented during LR parsing. **(9)**

2. (a) What is an Intermediate Representation (IR)? What are the advantages of using an IR? Give two examples of IR. **(8)**

(b) Write syntax-directed definition (SDD) for the following flow-of-control statement corresponding to a **for** loop in 'C'. **(9)**

$S \rightarrow$ **for** $(S_1; B; S_2)\ S_3$

Assume S has an inherited attribute *next* that represents a label assigned to the next statement following S. The non-terminal B represents a Boolean expression.

Contd …… P/2

CSE 309
Contd... Q. No. 2

(c) Write down a translation scheme for generating code using *backpatching* for the flow-of-control statements yielded by the following productions. You may add market symbols in the productions as required. Make other justified assumptions as required. **(12)**

(i) $S \rightarrow$ **if** (B) S_1 **else** S_2

(ii) $\rightarrow$ **while** (B) S_1

(d) translate the following if-else statement to three-address code avoiding redundant gotos. Assume &&, || are left associative and && has higher precedence than ||. **(6)**

if ((x! = y || x > 20) && x < 10) flag = 0; else flag = 1;

3. (a) The baker's mark-and-sweep algorithm for garbage collection moves objects among four lists: *Free, Unreached, Unscanned,* and *Scanned.* Illustrate how Baker's mark-and-sweep algorithm garbage collector works using the example network below when the pointer A → D is deleted. Assume X and Y are members of the root set. What is the time and space complexity of the algorithm? **(15)**

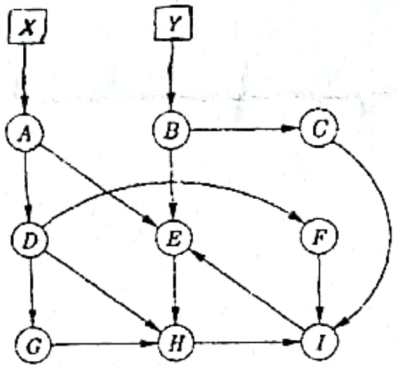


Figure for Question 3(a)

(b) Explain why register allocation and assignment is a critical issue during code generation? While generating code for a basic block, a function is used to select registers for each memory location associated with a three-address instruction *I*. Assume the function is called *getReg(I)*. For an instruction of the form, *I*: $x = y + z$, what rules are used by the function *getReg(I)* while selecting registers for each of the variables, *x*, *y*, and *z*. **(5+15=20)**

4. (a) Consider the three-address code below and answer the following questions.

(i) Identify the basic blocks and construct the flow graph for the given code. Give each block an identifier and replace the label references in the statements with the corresponding identifiers. Also, find and list the loops in the flow graph. **(14)**

Contd P/3

CSE 309
Contd... Q. No. 4(a)

(ii) Optimize the code by using all possible semantic preserving transformations. For each of the transformations used, specify the name of the transformation. **(14)**

```
1     L1: i = 0
2     L3: t1 = n - 1
3         iffalse i < t1 goto L2
4     L4: j = i + 1
5     L5: iffalse j < n goto L6
6     L7: t2 = i * 8
7         t3 = a [ t2 ]
8         t4 = j * 8
9         t5 = a [ t4 ]
10        iffalse t3 > t5 goto L8
11    L9: t6 = i * 8
12        p = a [ t6 ]
13    L10:  t7 = i * 8
14        t8 = j * 8
15        t9 = a [ t8 ]
16        a [ t7 ] = t9
17    L11:  t10 = j * 8
18        a [ t10 ] = p
19    L8: j = j + 1
20        goto L5
21    L6: i = i + 1
22        goto L3
23    L2:
```

(b) Consider a 'C' code fragment and the corresponding assembly code generated by some 'gcc' compiler using -O2 optimization switch. Now, identify the optimizations that have been performed by the compiler for the given 'C' code. Write an equivalent 'C' program for the optimized code. **(7)**

CSE 309

Contd... Q. No. 4(b)

Program written in C

```
#include<stdio.h>
int fact(int n)
{
    if (n == 0)  return 1;
    return n*fact(n-1);
}
int main(){
    int i=fact(5);
    printf("%d",i);
}
```

Assembly code generated by gcc

```
fact(int):
  mov eax, 1
  test edi, edi
  je .L1
.L2:
  imul eax, edi
  sub edi, 1
  jne .L2
.L1:
  ret
.LC0:
  .string "%d"
main:
  sub rsp, 8
  mov esi, 120
  mov edi, OFFSET FLAT:.LC0
  xor eax, eax
  call printf
  xor eax, eax
  add rsp, 8
  ret
```

SECTION – B

There are **FOUR** questions in this section. Answer any **THREE.**

5. (a) Describe the working principal of the language processor used by Java Programming Language? **(10)**

(b) Suppose you want to run a company that will produce compilers for different programming languages. If you need to build compilers for 10 programming languages, then which approach you will follow to build the front-end and back-end? Give a high-level overview of your design. **(15)**

[Your design must be efficient]

(c) In C programming language special keywords fall under the category of identifiers. Explain how lexical analyzer differentiates special keywords from identifiers. **(10)**

6. (a) Perform left factoring for the following grammar. **(5)**

$S \rightarrow abA \mid abcS$

$A \rightarrow aA \mid \in$

Contd P/5

CSE 309
Contd... Q. No. 6

(b) Briefly describe the following questions. **(5+8+7=20)**

i. Is it possible to eliminate lexical analyzer step from compilation?
(just write Yes or No)

ii. Why lexical analyzer uses buffer pair instead of single buffer?

iii. What does it mean by 'Panic Mode' in the context of lexical analyzer?

(c) Does lexical analyzer will find errors from the following code snippet? If you think lexical analyzer will find some errors then explain them properly. **(10)**

```
int arr[] = {1, 2, 3, 5};
int i = 0;
do {
    if (arr[i] %2 == 0)
        print("EVEN\n");
    else
        print("ODD\n");
    i++;
 } while (index < 4);
```

7. (a) Explain the difference among the following regular expressions in the context of Lex lexical analyzer generator. **(14)**

i. {mytoken}

ii. [mytoken]

iii. mytoken

iv. (mytoken)+

v. mytoken +

vi. (mytoken)*

vii. mytoken*

(b) Perform left recursion elimination from the following grammar. **(21)**

$S \rightarrow Sa \mid Tb \mid Uc \mid a$

$T \rightarrow Sc \mid Ta \mid Ub \mid b$

$U \rightarrow Sb \mid Tc \mid Ua \mid c$

8. (a) Compute FIRST and FOLLOW for the following grammar. **(5+10=15)**

$T \rightarrow S\$$

$S \rightarrow DbE$

$D \rightarrow VD \mid \epsilon$

$V \rightarrow cdE \mid acdE$

$E \rightarrow c \mid \epsilon$

Contd P/6

CSE 309

Contd... Q. No. 8

(b) Consider the following grammar. **(20**

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \in$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \in$

$F \rightarrow (E) \mid id$

On input id + id * id, construct a table by showing the moves that will be made by the non-recursive predictive parser algorithm.

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY, DHAKA

L-3/T-1 B. Sc. Engineering Examinations (January 2021 Term)

Sub: **CSE 309** (Compiler)

Full Marks: 210, Section Marks: 105, Time: 2 Hours 30 Minutes (Sections A + B)

USE SEPARATE SCRIPTS FOR EACH SECTION

The figures in the margin indicate full marks.

SECTION – A

There are **NINE** questions in this section. Answer any **SEVEN**.

1. Explain the concept of "Line Control Directive" in compilation with necessary examples. (15)

2. Elaborate the role of a linker in compilation with reference to the relocatable machine code. (15)

3. Explain clearly, the relative merits and demerits of using: (7 + 8)

   i. The same token with appropriate attributes for the comparison operators $<$, $\leq$, $>$, $\geq$, $\neq$, and $==$.

   ii. The same token with appropriate attributes for the arithmetic operators $+$, $-$, $\times$, and $\div$.

4. (a) Elucidate how predictive parsers ensure that backtracking is avoided. (10)

   (b) Write down the rule for using the "ϵ-productions" in predictive parsing. (5)

5. Eliminate left recursion from the following grammar: (15)

$$X \rightarrow XSb \mid Sa \mid b$$
$$S \rightarrow Sb \mid Xa \mid a$$

   You need to show the steps.

6. (a) Discuss the relative advantages and disadvantages of using single buffer instead of using buffer pairs in lexical analysis. (6)

   (b) Consider the two grammars given below. (9)

   Grammar 1

$$A \rightarrow aA \mid bd$$
$$B \rightarrow aB \mid cc$$

   Grammar 2

$$A \rightarrow aA \mid ad$$
$$B \rightarrow bB \mid cc$$

   Both of the above mentioned grammars have at least two bodies of the production rules beginning with an a. Explain in detail with proper justifications why one of the grammars is LL(1) while the other is not.

Contd P/2

CSE 309

7. In carrying out nonrecursive predictive parsing for an input string w and a parsing table M, if X is the top stack symbol, a is the current input symbol, and $M[X, a] = X \to Y_1 Y_2 \ldots Y_k$, the following actions are carried out, (15)

   (i) output the production $X \to Y_1 Y_2 \ldots Y_k$,

   (ii) pop the stack,

   (iii) push $Y_k, Y_{k-1}, \ldots, Y_2, Y_1$ onto the stack, with Y_1 on top.

   Explain with necessary figures and justifications, how the actions above correspond to top-down left-most derivation.

8. Define a "handle" in LR parsing. For the grammar, (15)

   $S \to aAcBe$

   $A \to Ab \mid b$

   $B \to d$

   and the input string $abbcde$, identify the handles and show how these are being pruned for a case of LR parsing. You need to construct a table with the columns Stack, Input, and Action as a part of your answer.

9. Augment the modified expression grammar given below and hence construct the initial set (often indicated as I_0) of LR(1) items. (15)

   $E \to E + T \mid T$

   $T \to T * F \mid F$

   $F \to$ **id**

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY, DHAKA

L-3/T-1 B. Sc. Engineering Examinations 2019-2020

Sub: **CSE 309** (Compiler)

Full Marks: 210 Section Marks: 105 Time: 2 Hours 30 Minutes (Sections A + B)

USE SEPARATE SCRIPTS FOR EACH SECTION

The figures in the margin indicate full marks.

SECTION – B

There are **FOUR** questions in this section. Answer any **THREE**.
Symbols have their usual meanings in the context of your text unless otherwise specified.

10. (a) Below is a grammar for variable declarations of the forms such as `int i, k[2], j[2][3];` Now write an SDT to save the type and width information of all the declared identifiers in the symbol table. Assume that (i) types are represented as type expressions and (ii) widths of an integer and a float are 4 and 8 bytes, respectively. Make other justified assumptions as required. **(12)**

$V \rightarrow T\ D$ **;**
$T \rightarrow$ **int | float**
$D \rightarrow D_1$ **,** **id** C | **id** C
$C \rightarrow$ **[num]** C_1 | ϵ

(b) Below is an SDT for generating three-address code for array references. What will be the three-address code for the expression `a[b[i]][c[j]]` generated by the given SDT? Assume that the arrays a, b and c are declared as, **`int`** `a[5][5], b[3], c[3]` and types and widths of the arrays have correctly been saved in the symbol table. The attributes associated with the terminals and nonterminals have their usual meanings. Make other justified assumptions as required. **(11)**

$L \rightarrow$ **id** *[E]* { *L.array = top.get(***id***.lexeme);*
L.type = L.array.type.elem;
L.addr = **new** *Temp();*
gen(L.addr '=' E.addr '' L.type.width); }*

| L_1 *[E]* { *L.array =* L_1*.array;*
L.type = L_1*.type.elem;*
t = **new** *Temp();*
L.addr = **new** *Temp();*
gen(t '=' E.addr '' L.type.width);*
gen(L.addr '=' L_1*.addr '+' t); }*

$E \rightarrow$ **id** { *E.addr = top.get(***id***.lexeme); }*

| L { *E.addr =* **new** *Temp();*
gen(E.addr '=' L.array.base '[' L.addr ']'); }

(c) The following SDT prints the prefix form of an expression, i.e., performs infix to prefix translation. Explain why it is not possible to implement the SDT during either top-down or bottom-up parsing. Describe, then, how such an SDT can be implemented. **(6+6)**

$L \rightarrow E$ **n**
$E \rightarrow$ { print('+'); } $E_1 + T$
$E \rightarrow T$
$T \rightarrow$ { print('*'); } $T_1 * F$
$T \rightarrow F$
$F \rightarrow (E)$
$F \rightarrow$ **digit** { print(**digit.level**); }

11. Consider the three-address code below and answer the following questions.

```
1   L1:  i = 0
2   L3:  d = 4
3   L4:  iffalse i < d goto L2
4   L5:  j = i + 1
5   L6:  iffalse j < d goto L7
6   L8:  t1 = i * 16
7        t2 = j * 4
8        t3 = t1 + t2
9        temp = a [ t3 ]
10  L9:  t4 = i * 16
11       t5 = j * 4
12       t6 = t4 + t5
13       t7 = j * 16
14       t8 = i * 4
15       t9 = t7 + t8
16       t10 = a [ t9 ]
17       a [ t6 ] = t10
18  L10: t11 = j * 16
19       t12 = i * 4
20       t13 = t11 + t12
21       a [ t13 ] = temp
22  L11: j = j + 1
23       goto L6
24  L7:  i = i + 1
25       goto L4
26  L2:
```

(a) Identify the basic blocks and construct the flow graph for the given code. **(17)**

(b) Optimize the code by using applicable transformations from the following list. (i) common subexpression elimination, (ii) copy propagation, (iii) dead code elimination, (iv) reduction in strength on induction variables, (v) elimination of induction variables. **(18)**

12. (a) What is backpatching? Write down a translation scheme (semantic actions) for generating code using backpatching for boolean expressions yielded by the following productions. You may add marker symbols in the productions as required. **(3+9)**

$B \rightarrow B_1 \,||\, B_2$

$B \rightarrow B_1 \,\&\&\, B_2$

$B \rightarrow E_1$ **rel** E_2

(b) Based on the SDT in Question 12(a) above, draw the annotated parse tree for the expression: x != y && x > 20 II x < 100. Assume that the operators II and && are left-associative and && has higher precedence than II. Assume that the starting instruction number is 200. **(10)**

(c) Consider the network of objects below. Suppose at some point in time, the pointer A → C is deleted. Then we perform a mark-and-compact garbage collector on the network. Also, suppose that, **(13)**

i. Each object has size 50 bytes, and
ii. Initially, all the nine objects in the heap are arranged in alphabetical order, starting at byte 0 of the heap.

What is the address of each object after garbage collection? Draw the heap showing locations of the allocated objects before and after running garbage collection.

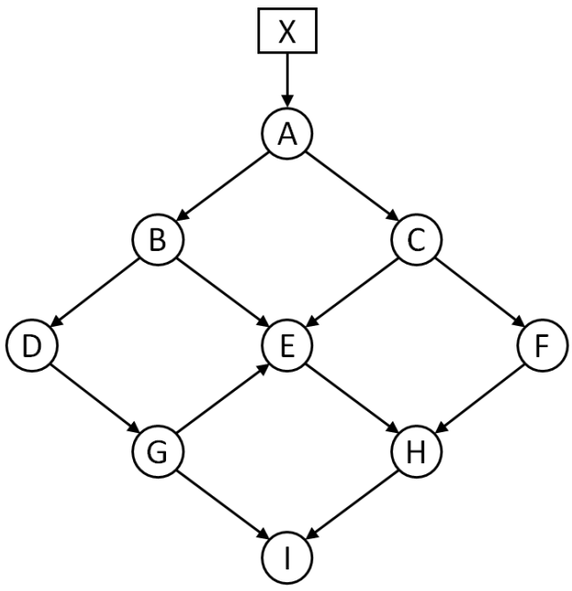


Figure for Question 12(c)

13. (a) Determine the liveness and next-use information for each statement in the following basic block. Also show the state of the symbol table at each of the steps of your algorithm. How does next-use information facilitate code optimization? Illustrate using the example of the given basic block. **(12+6)**

```
t = a + b
u = c + d
t = d - b
v = a + c
d = t + v
```

(b) Translate the following basic block consisting of three-address statements into machine-code instructions for the simple machine model of your text. Assume that two registers are available. Make sure the registers are allocated and assigned efficiently to avoid unnecessary loads and stores. Show the register and address descriptors after each step. Assume p and q are temporary variables and the remaining variables are live on exit from the block. **(17)**

```
p = a + b
q = c - p
c = d
d = q - c
exit
```